

Relatório Técnico de Segurança da Informação

Análise e Melhoria de Segurança em Sistema Web de Ocorrências Acadêmicas

Disciplina: Segurança da Informação

Professor: Edson Vaz Lopes

Curso: Engenharia de Software — Católica SC

Data: 05 de maio de 2026

Integrantes do grupo: Henrique Cordeiro de Oliveira, Lucas Rogério Mendonça e Nicholas Scoz dos Santos

Turma: Engenharia de Software

Link do repositório: <https://github.com/nicholasscoz/seguranca-da-informacao-ap01>

Link do sistema publicado: <https://nicholasscoz.github.io/seguranca-da-informacao-ap01/>

1. Descrição Resumida do Sistema

O sistema analisado é um protótipo web para registro de ocorrências acadêmicas em uma instituição de ensino. Ele permite simulação de login com diferentes perfis (aluno, professor, administrador), cadastro de ocorrências vinculadas a alunos, listagem e busca de registros, alteração de status, exportação de dados e visualização de logs de auditoria.

A aplicação foi construída exclusivamente com HTML, CSS e JavaScript, sem back-end, API ou banco de dados real. Todos os dados são armazenados no localStorage do navegador do usuário. O sistema manipula dados pessoais simulados como nome, matrícula, CPF, e-mail pessoal, telefone e descrições de ocorrências.

O sistema não possui autenticação real, autorização validada no servidor, controle de sessão seguro, persistência centralizada ou qualquer mecanismo de proteção em camada de servidor.

2. Regras de Negócio Percebidas

No protótipo original, foram identificadas as seguintes regras de negócio, a maioria apenas simuladas no front-end:

1. O sistema possui três perfis: aluno, professor e administrador, mas na versão original não há diferenciação real de permissões.
2. Qualquer usuário logado pode cadastrar, editar status, excluir ocorrências e exportar todos os dados, independente do perfil.
3. O login aceita credenciais definidas em texto puro no código JavaScript, com senhas fracas (123456 e admin).
4. O usuário pode alterar seu próprio perfil livremente por meio de um seletor na interface, sem qualquer validação.
5. A busca percorre todos os campos (incluindo CPF e e-mail pessoal), expondo dados sensíveis a qualquer perfil.
6. A exportação inclui senhas dos usuários, token de API, cópia completa do localStorage e todos os logs.
7. Os logs podem ser apagados por qualquer perfil, sem confirmação e sem registro da limpeza.
8. Não há expiração de sessão, permitindo que o sistema fique logado indefinidamente.
9. Não existe validação dos campos do formulário de ocorrência (campos vazios são aceitos).
10. Os dados iniciais incluem CPF, e-mail pessoal e telefone dos alunos, coletados sem necessidade funcional clara.

3. Identificação dos Ativos

Item	Ativo Identificado	Por que este ativo tem valor?
1	Dados dos alunos (nome, matrícula)	Dados pessoais protegidos por LGPD; vazamento gera dano à privacidade e responsabilidade legal.
2	Registros de ocorrências	Contêm informações acadêmicas e comportamentais; podem causar dano moral se expostos.
3	Credenciais de acesso (e-mails e senhas)	Permitem acesso ao sistema; expostas no código-fonte, permitem impersonação.

Item	Ativo Identificado	Por que este ativo tem valor?
4	Logs de auditoria	Permitem rastreabilidade de ações; sua manipulação compromete a capacidade de auditoria.
5	Código-fonte (HTML/CSS/JS)	Contém toda a lógica do sistema, credenciais e regras de negócio; acessível a qualquer usuário.
6	Token de API exposto	Simula credencial de integração; em produção, permitiria acesso não autorizado a serviços.
7	Dados exportados (JSON)	Contém cópia completa dos dados do sistema incluindo senhas e tokens.
8	Dados no localStorage	Persistência local manipulável pelo usuário; sem proteção contra leitura ou alteração.

4. Classificação dos Dados e Ativos

Item	Dado ou Ativo	Classificação	Justificativa
1	Nome e matrícula do aluno	Interno	Necessários para operação, mas não devem ser públicos.
2	CPF do aluno	Confidencial	Dado pessoal sensível (LGPD); desnecessário para o sistema.
3	Senhas dos usuários	Restrito	Acesso deve ser exclusivo do titular; jamais visíveis em código.
4	Observações internas	Confidencial	Informações administrativas que não devem ser vistas por alunos.
5	Logs de auditoria	Interno	Necessários para rastreabilidade; devem ser protegidos contra alteração.
6	Token de API	Restrito	Credencial de integração; exposição permite acesso não autorizado.
7	Dados exportados	Confidencial	Arquivo contém todos os dados do sistema, incluindo credenciais.
8	Descrição das ocorrências	Interno	Informações operacionais que devem ser visíveis conforme perfil.

5. Análise dos Riscos Encontrados

Item	Problema	Risco	Impacto
1	Senhas em texto puro no JS	Violação de confidencialidade	Qualquer usuário pode ver as credenciais pelo DevTools, acessando qualquer perfil.
2	Ausência de controle de acesso (RBAC)	Violação de autorização	Aluno pode excluir ocorrências, exportar dados e limpar logs como se fosse admin.
3	Troca livre de perfil pelo usuário	Escalação de privilégio	Usuário altera perfil pelo seletor ou console, assumindo permissões de administrador.
4	Coleta excessiva de CPF, telefone, e-mail pessoal	Violação da LGPD (minimização)	Dados pessoais sensíveis coletados sem necessidade funcional, aumentando risco de vazamento.
5	Token de API exposto no código	Vazamento de credencial	Em produção, permitiria acesso não autorizado a serviços externos.
6	Exportação inclui senhas e localStorage	Vazamento massivo de dados	Arquivo exportado contém todas as credenciais e dados do sistema.
7	Logs manipuláveis e apagáveis por qualquer perfil	Perda de rastreabilidade	Impossibilita auditoria; ações maliciosas podem ser ocultadas.
8	Sem expiração de sessão	Acesso indevido por sessão abandonada	Computador compartilhado mantém acesso indefinidamente.
9	Sem sanitização de entradas (XSS)	Injeção de código malicioso	Dados inseridos são renderizados sem tratamento, permitindo execução de scripts.
10	Sem validação de campos obrigatórios	Integridade dos dados comprometida	Registros podem ser criados vazios ou com dados inválidos.
11	Dados pessoais visíveis para todos os perfis	Violação de confidencialidade	Aluno vê CPF, telefone e observações internas de outros alunos.
12	Senhas fracas (123456, admin)	Facilidade de acesso não autorizado	Mesmo com autenticação real, senhas fracas são trivialmente descobertas.

6. Melhorias Implementadas no Código

Item	Melhoria	Arquivo	O que mudou
1	Controle de acesso por perfil (RBAC)	app.js	Criado objeto PERMISSIONS com permissões por perfil. Cada ação verifica hasPermission() antes de executar. Seções da interface são exibidas/ocultadas conforme perfil.
2	Remoção do seletor livre de perfil	index.html, app.js	Removido o select que permitia trocar de perfil. O perfil agora é fixo conforme o login realizado.
3	Sanitização contra XSS	app.js	Implementada função sanitizeHTML() que usa textContent/innerHTML para escapar caracteres especiais. Aplicada em toda renderização dinâmica.
4	Validação de campos obrigatórios	app.js, index.html	Campos obrigatórios têm atributo required e maxlength. Validação programática em JavaScript antes de salvar.
5	Remoção de CPF, e-mail pessoal e telefone	index.html, app.js	Campos de CPF, e-mail pessoal e telefone removidos do formulário e dos dados iniciais. Princípio de minimização de dados (LGPD).
6	Expiração automática de sessão	app.js	Sessão expira após 15 minutos. Timer visual na interface. Renovação automática a cada interação do usuário.
7	Bloqueio por tentativas de login	app.js	Após 5 tentativas falhas, login é bloqueado por 2 minutos. Mensagem genérica não revela se é e-mail ou senha incorretos.
8	Exportação segura (sem senhas/tokens)	app.js	Exportação inclui apenas ocorrências. Removidos: array USERS, FAKE_API_TOKEN, cópia do localStorage.
9	Remoção do token de API do código	app.js	Variável FAKE_API_TOKEN removida. Em produção, tokens devem ser gerenciados pelo servidor.
10	Logs protegidos e restritos ao admin	app.js, index.html	Seção de logs visível apenas para administradores. Limpeza requer confirmação e registra a ação. Logs limitados a 500 entradas.
11	Confirmação para ações destrutivas	app.js	Exclusão de ocorrências, limpeza de logs e restauração de dados exigem confirmação via confirm().
12	Senhas mais fortes na demonstração	app.js	Senhas alteradas de '123456'/admin' para 'Demo@2026'/Admin@2026' (simulação de política de senhas).
13	Banner de aviso de protótipo	index.html, style.css	Adicionado banner permanente informando que o sistema é um protótipo e que dados reais não devem ser inseridos.
14	Metadados de segurança no HTML	index.html	Adicionados meta tags: X-Content-Type-Options, X-Frame-Options, referrer policy. Efetivos apenas com servidor real.
15	Observações internas restritas ao admin	app.js, index.html	Campo de observação interna visível apenas para administradores no formulário e na listagem.
16	Senhas removidas da tela de login	index.html	Senhas não são mais pré-preenchidas nos campos do formulário. Removido o display em texto das senhas na lista de usuários de demonstração.
17	Logs não expõem dados pessoais	app.js	Logs registram apenas matrícula e ID da ocorrência, sem incluir CPF, descrição completa ou JSON do registro.

7. Relação entre Problema, Risco, Controle e Justificativa

#	Problema	Risco Associado	Controle Proposto	Implementado?
1	Senhas em texto puro no código-fonte	Qualquer pessoa pode ver credenciais pelo DevTools (confidencialidade)	Senhas reforçadas na demo; recomendação de hash no servidor	Parcialmente
2	Ausência de RBAC	Qualquer perfil executa qualquer ação (autorização)	Implementado objeto PERMISSIONS com verificação em cada função	Sim
3	Seletor livre de perfil	Escalação de privilégio trivial (autenticidade)	Removido seletor; perfil fixo ao login	Sim
4	Coleta excessiva de dados pessoais	Violação do princípio de minimização da LGPD (privacidade)	Removidos CPF, e-mail pessoal e telefone do formulário e dados	Sim
5	Token de API exposto	Vazamento de credencial de integração (confidencialidade)	Variável removida do código; tokens devem ficar no servidor	Sim
6	Exportação insegura	Vazamento massivo de senhas e dados (confidencialidade)	Exportação restrita a ocorrências; sem senhas, tokens ou localStorage	Sim
7	Logs sem proteção	Perda de rastreabilidade e capacidade de auditoria (integridade)	Logs restritos ao admin; limpeza com confirmação e registro	Parcialmente
8	Sem expiração de sessão	Acesso indevido em computadores compartilhados (controle de acesso)	Sessão com timeout de 15 min; timer visual; renovação por interação	Sim
9	Vulnerabilidade a XSS	Injeção de código malicioso via campos do formulário (integridade)	Sanitização com sanitizeHTML() em toda renderização	Sim
10	Sem validação de campos	Registros inválidos ou vazios comprometem integridade dos dados	Validação obrigatória em JS e atributos HTML (required, maxlength, pattern)	Sim
11	Dados visíveis para todos os perfis	Violação de confidencialidade e menor privilégio	Aluno vê apenas próprias ocorrências; obs. internas restritas ao admin	Sim
12	Senhas fracas	Acesso não autorizado por força bruta (autenticação)	Senhas mais complexas na demo; bloqueio após 5 tentativas	Parcialmente

#	Problema	Risco Associado	Controle Proposto	Implementado?
13	Mensagem de erro revela informação	Enumeração de usuários por mensagens diferenciadas (confidencialidade)	Mensagem genérica: 'Credenciais inválidas' sem indicar campo incorreto	Sim
14	Sem aviso de limitação do protótipo	Usuário pode confiar dados reais a um sistema sem proteção	Banner permanente alertando que é protótipo didático	Sim
15	Logs expõem dados pessoais completos	Violação de privacidade em registros de auditoria	Logs registram apenas matrícula e ID, sem dados pessoais extensos	Sim

8. Justificativa Técnica das Decisões

Item 1 — Senhas em texto puro

As credenciais estavam definidas como strings no array `USERS` do JavaScript, visíveis por qualquer pessoa no DevTools. Isso viola o princípio da confidencialidade. As senhas foram reforçadas para demonstração, mas em produção devem ser armazenadas como hashes (bcrypt/argon2) no servidor. Conceito: confidencialidade, autenticação segura.

Item 2 — Implementação de RBAC

Na versão original, todos os perfis tinham acesso idêntico a todas as funcionalidades, violando o princípio do menor privilégio. Foi implementado um objeto `PERMISSIONS` que define capacidades por perfil, e cada função verifica `hasPermission()` antes de executar. Conceito: RBAC, princípio do menor privilégio, autorização.

Item 3 — Remoção do seletor de perfil

O seletor permitia que qualquer usuário assumisse perfil de administrador, configurando escalação de privilégio trivial. A remoção impede essa alteração pela interface, embora via console ainda seja possível (limitação do front-end). Conceito: autenticidade, controle de acesso.

Item 4 — Minimização de dados (LGPD)

O formulário coletava CPF, e-mail pessoal e telefone sem necessidade funcional clara. A LGPD (Art. 6, III) exige que a coleta de dados pessoais se limite ao mínimo necessário. Esses campos foram removidos. Conceito: minimização de dados, LGPD, privacidade.

Item 5 — Token de API removido

A variável `FAKE_API_TOKEN` ficava exposta no código-fonte. Em produção, tokens de API permitem acesso a serviços e nunca devem ficar no front-end. A variável foi removida. Conceito: confidencialidade, posse, dados em repouso.

Item 6 — Exportação segura

A exportação original incluía o array `USERS` (com senhas), o token de API e cópia completa do `localStorage`. A nova versão exporta apenas as ocorrências, sem dados sensíveis. Conceito: confidencialidade, classificação de dados.

Item 7 — Proteção de logs

Os logs podiam ser apagados por qualquer perfil sem confirmação. Na versão melhorada, apenas o administrador pode visualizar e limpar logs, com confirmação obrigatória e registro da limpeza. Limitação: logs no `localStorage` ainda podem ser manipulados pelo console. Conceito: rastreabilidade, auditoria, integridade.

Item 8 — Expiração de sessão

Na versão original, a sessão permanecia ativa indefinidamente. Em ambientes compartilhados, isso permite acesso indevido. Foi implementado `timeout` de 15 minutos com renovação por interação e timer visual. Conceito: controle de sessão, disponibilidade.

Item 9 — Sanitização contra XSS

A renderização dinâmica usava `template literals` sem sanitização, permitindo injeção de HTML/JavaScript malicioso. Foi implementada a função `sanitizeHTML()` que converte caracteres especiais em entidades

HTML. Conceito: integridade, dados em uso, validação de entrada.

Items 10-15 — Validação, visibilidade, mensagens, banner, logs

Cada melhoria segue princípios complementares: validação protege integridade; visibilidade por perfil respeita confidencialidade; mensagens genéricas de erro evitam enumeração de usuários; o banner informa limitações do protótipo; e logs sem dados pessoais respeitam privacidade. Conceitos: integridade, confidencialidade, LGPD, transparência.

9. Melhorias que Dependem de Back-end, Banco de Dados ou Infraestrutura

#	Melhoria Necessária	Depende de	Por que não pode ser resolvida no front-end?
1	Autenticação real com hash de senhas	Back-end	Senhas precisam ser comparadas como hashes no servidor. No front-end, qualquer comparação é visível e contornável.
2	Autorização validada no servidor	Back-end	Permissões verificadas no front-end podem ser alteradas pelo console. O servidor deve validar cada requisição.
3	Sessão com token seguro (JWT/cookie httpOnly)	Back-end	Sessão no localStorage é manipulável. Tokens seguros exigem emissão e validação pelo servidor.
4	Banco de dados centralizado	Banco de dados	localStorage é local, volátil e sem backup. Dados acadêmicos exigem persistência confiável e centralizada.
5	Logs persistentes e imutáveis	Back-end + BD	Logs no localStorage podem ser apagados pelo usuário. Logs de auditoria devem ser armazenados no servidor de forma append-only.
6	HTTPS obrigatório	Infraestrutura	Dados em trânsito precisam de criptografia TLS. Sem HTTPS, credenciais e dados trafegam em texto claro.
7	Backup e recuperação de dados	Infraestrutura	Não existe backup dos dados no localStorage. Estratégia de backup exige infraestrutura de servidor.
8	Política de senhas com complexidade e expiração	Back-end	Regras de complexidade e rotação de senhas devem ser aplicadas no servidor, não no front-end.
9	Criptografia de dados em repouso	Infraestrutura + BD	Dados sensíveis no banco devem ser criptografados. No localStorage, são texto puro acessível.
10	Controle de acesso baseado em contexto (ABAC)	Back-end	Políticas que consideram horário, localização e contexto exigem avaliação no servidor.

10. Limitações da Solução Entregue

1. Todas as proteções implementadas são simulações no front-end e podem ser contornadas pelo usuário via DevTools ou console do navegador.
2. As senhas continuam no código-fonte JavaScript, mesmo com complexidade maior; em produção, devem ser hashes no servidor.
3. A sessão com timeout é armazenada no localStorage e pode ser manipulada para estender ou forjar sua validade.
4. Os dados persistem apenas no navegador local. Não há sincronização entre dispositivos, backup ou proteção contra limpeza do navegador.
5. O RBAC implementado é apenas visual e programático no front-end; qualquer verificação pode ser removida editando o JavaScript.
6. Os logs de auditoria podem ser editados ou apagados diretamente no localStorage pelo console.
7. Não há criptografia dos dados em repouso (localStorage armazena tudo em texto puro).
8. Não há proteção dos dados em trânsito, pois não há comunicação com servidor.
9. A sanitização contra XSS protege a renderização, mas não substitui validação e sanitização no servidor.
10. A versão publicada no GitHub Pages não possui HTTPS configurável pelo desenvolvedor (depende do GitHub).

11. Conclusão

O sistema, da forma como foi recebido originalmente, NÃO poderia ser usado em produção com dados reais.

Os principais impedimentos são: ausência total de back-end para autenticação e autorização reais; credenciais expostas em texto puro no código-fonte; ausência de banco de dados para persistência segura; coleta excessiva de dados pessoais em violação à LGPD; ausência de RBAC efetivo; logs manipuláveis; e vulnerabilidades como XSS e ausência de validação de entradas.

As melhorias implementadas nesta versão reduzem significativamente os riscos dentro das possibilidades de um protótipo front-end: RBAC simulado, sanitização, expiração de sessão, minimização de dados, exportação segura e proteção de logs. No entanto, essas medidas são apenas camadas de interface e podem ser contornadas.

Para que o sistema possa ser considerado apto para uso em produção, seria necessário no mínimo: implementar um back-end com autenticação baseada em hashes seguros; criar um banco de dados centralizado com controle de acesso; implementar autorização no servidor; configurar HTTPS; criar logs persistentes e imutáveis; e realizar uma adequação formal à LGPD com base legal para tratamento de dados pessoais de alunos.

O protótipo pode ser aproveitado como referência visual e funcional para o desenvolvimento de um sistema real, desde que toda a lógica de segurança, persistência e controle de acesso seja reimplementada em camada de servidor.